Chapter 6
Deadlocks

6.1 Resource
6.2 Introduction to deadlocks
6.3 The ostrich algorithm
6.4 Deadlock detection and recovery
6.5 Deadlock avoidance
6.6 Deadlock prevention
6.7 Other issues

Resources

- A **resource** is anything that can be used by a single process at any instant of time, and that must be acquired, used, and released over the course of time.
- Resource types R1, R2, . . ., Rm
  - ✓ CPU cycles, memory space, I/O devices ,record in a database…
- Each resource type Ri has Wi instances.
- Preemptable resources
  - ✓ can be taken away from a process with no ill effects (e.g. memory)
- Nonpreemptable resources
  - ✓ will cause the process to fail if taken away (e.g. CD recorder)

Resources

- Each process utilizes a resource as follows:
  1. request the resource
  2. use the resource
  3. release the resource
- Must wait if request is denied
  - requesting process may be blocked
  - may fail with error code

Resources

- Processes need access to resources in reasonable order
- Suppose a process holds resource A (e.g. scanner) and requests resource B (e.g. CD recorder)
  - at same time another process holds B and requests A
  - both are blocked and remain so
- Deadlocks occur when …
  - processes are granted exclusive access to devices

Using Semaphore to Share Resource

```
1 ⟹ Process P();
2 ⟹ { A.Down();
3 ⟹   B.Down();
        use both resource
4 ⟹   B.Up();
5 ⟹   A.Up(); }
```

```
1 ⟹ Process Q();
6 ⟹ { A.Down();
      B.Down();
        use both resource
      B.Up();
      A.Up(); }
```

```
1  External Semaphore A(1), B(1);
2  External Semaphore A(0), B(1);
3  External Semaphore A(0), B(0);
4  External Semaphore A(0), B(1);
5  External Semaphore A(1), B(1);
```

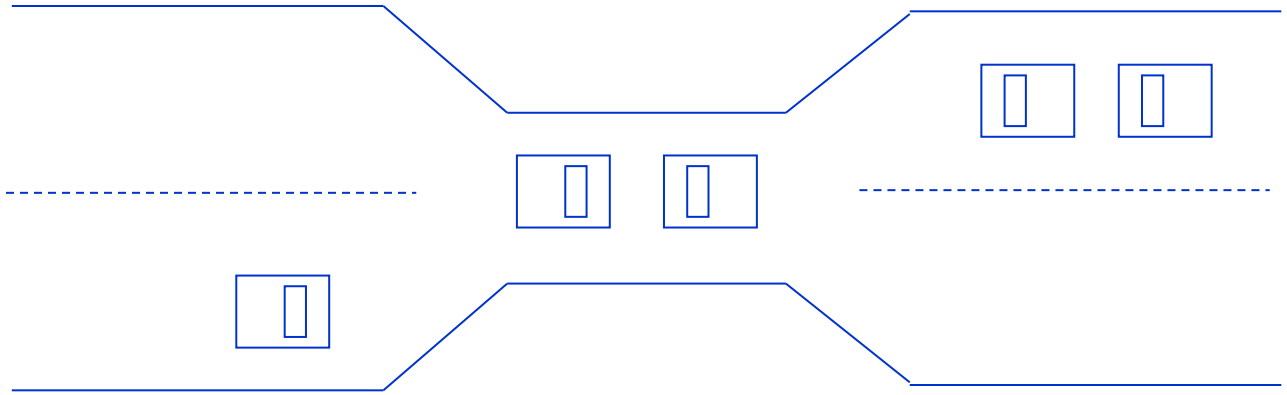
But Deadlock can Happen!

```
1 ⟹ Process P();                1 ⟹ Process Q();
2 ⟹ { A.Down();                 3 ⟹ { B.Down();
        B.Down();                       A.Down();
            use b                           use both resources
        B.Up()                          A.Up();
        A.Up()                          B.Up(); }
```




```
1   External Semaphore A(1), B(1);
2   External Semaphore A(0), B(1);
3   External Semaphore A(0), B(0);
```

Bridge Crossing Example



- Traffic only in one direction.
- Each section of a bridge can be viewed as a resource. If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback).
- Several cars may have to be backed up if a deadlock occurs.

Introduction to Deadlocks

- Formal definition :
  A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.
- Usually the event is release of a currently held resource
- None of the processes can …
  - run
  - release resources
  - be awakened

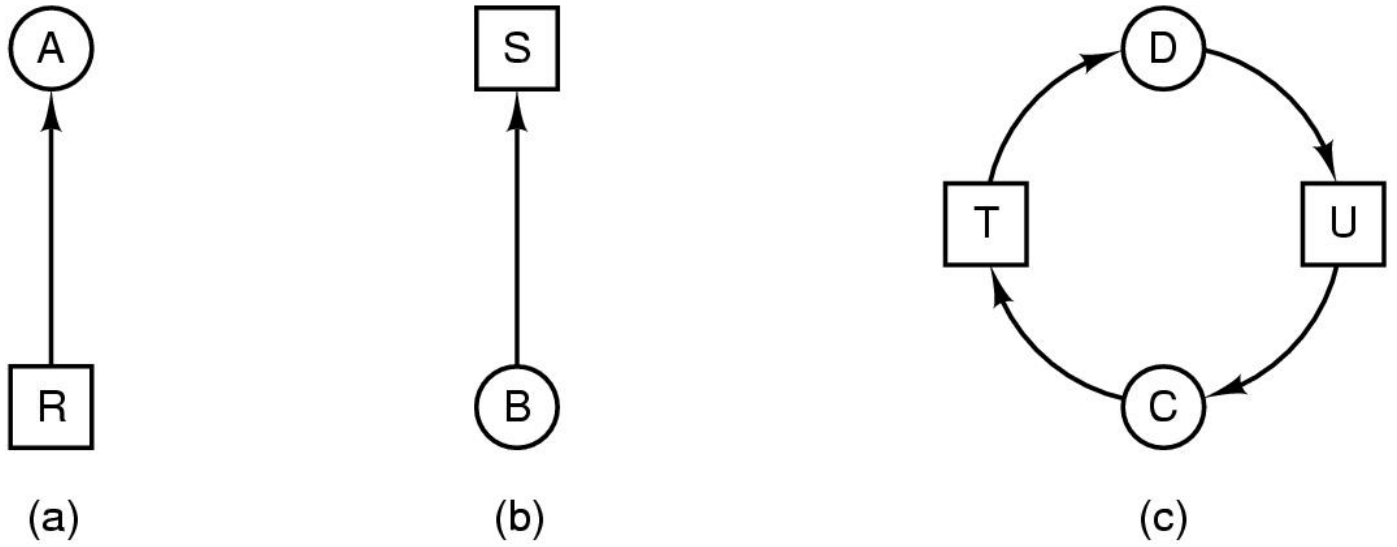
Four Conditions for Deadlock

Deadlock can arise if four conditions hold SIMULTANEOUSLY.

1. Mutual exclusion condition
   - each resource assigned to 1 process or is available
2. Hold and wait condition
   - process holding resources can request additional
3. No preemption condition
   - previously granted resources cannot forcibly taken away
4. Circular wait condition
   - must be a circular chain of 2 or more processes
   - each is waiting for resource held by next member of the chain

Deadlock Modeling

- Modeled with directed graphs



(a) (b) (c)

  - resource R assigned to process A
  - process B is requesting/waiting for resource S
  - process C and D are in deadlock over resources T and U

Resource-Allocation Graph

A set of vertices V and a set of edges E.

- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system.
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system.
- request edge – directed edge $P_i \rightarrow R_j$
- assignment edge – directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph

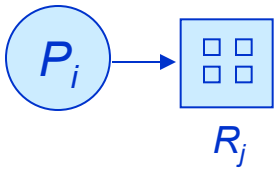
- Process



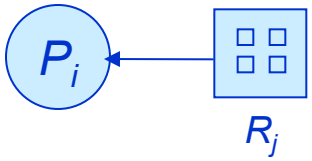
- Resource Type with 4 instances



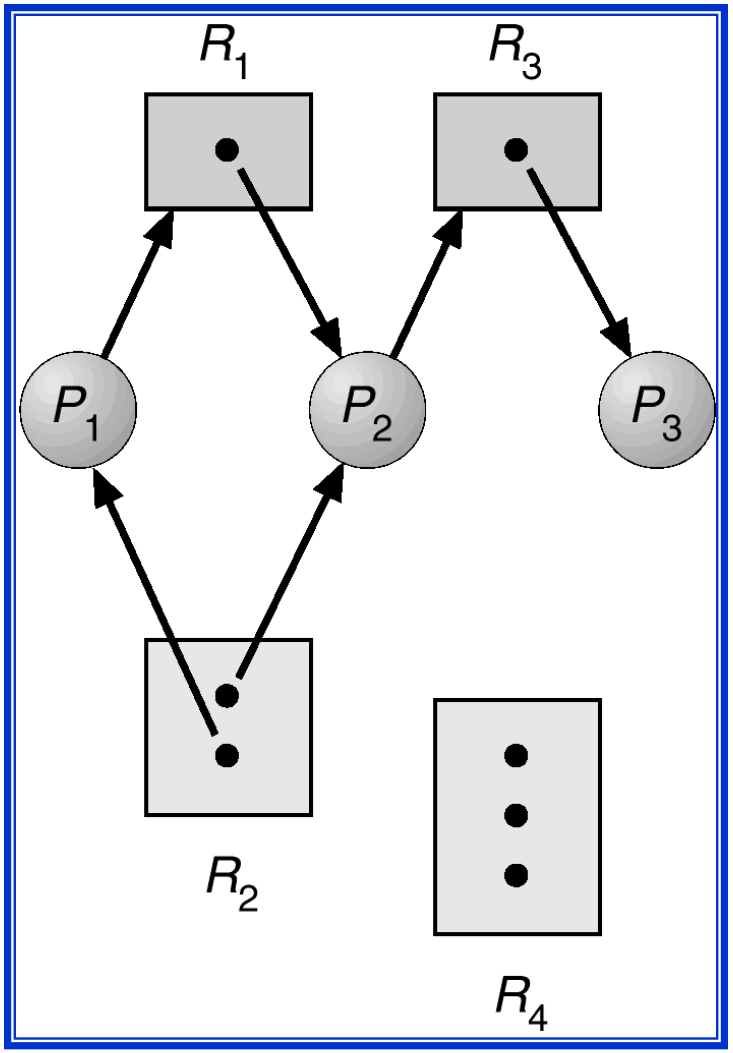
- P_i requests an instance of R_j



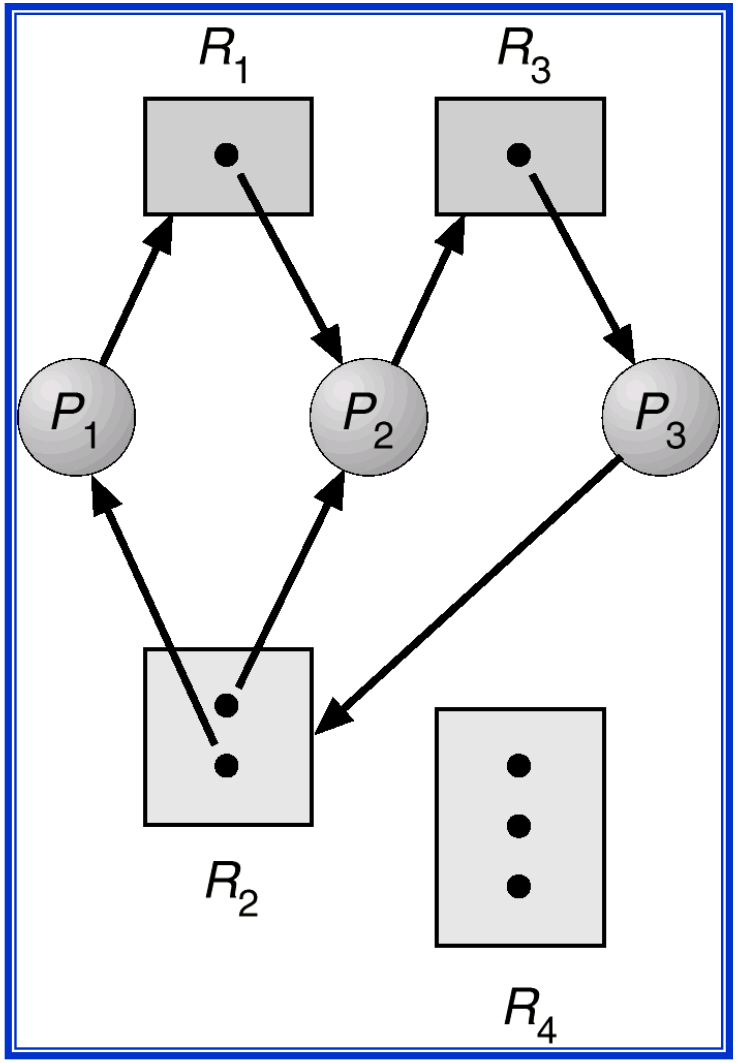
- P_i is holding an instance of R_j



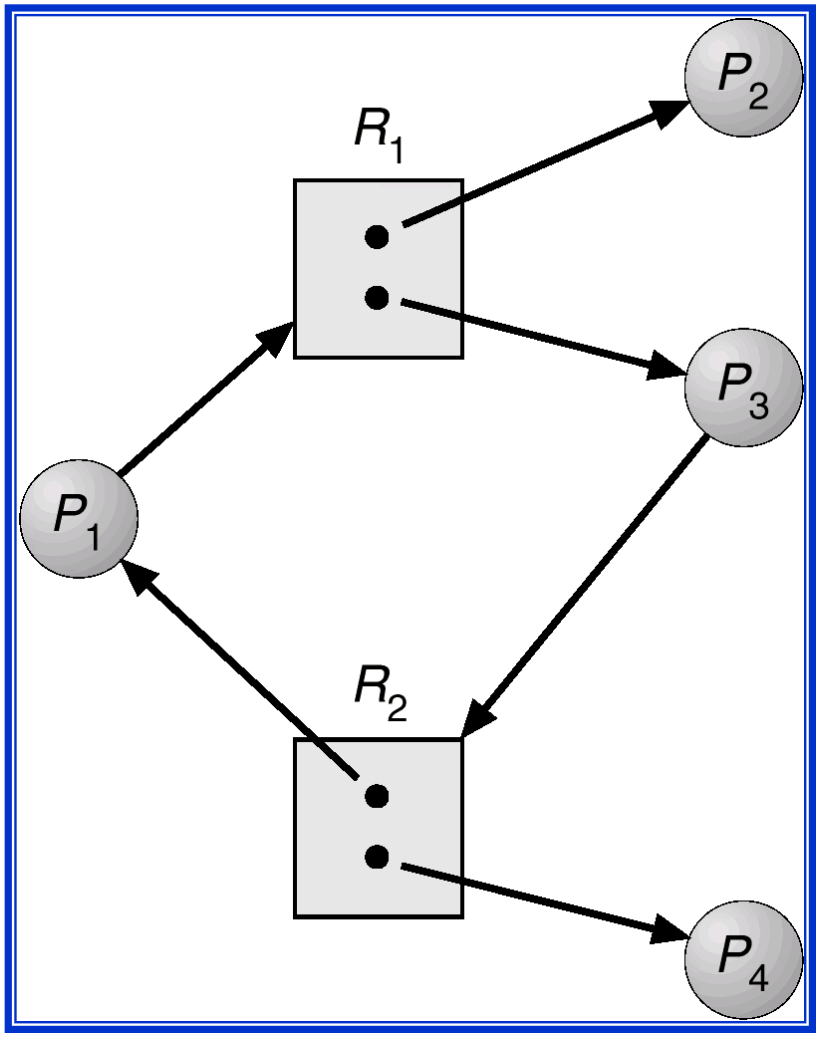
Example of a Resource Allocation Graph



Resource Allocation Graph With a Deadlock



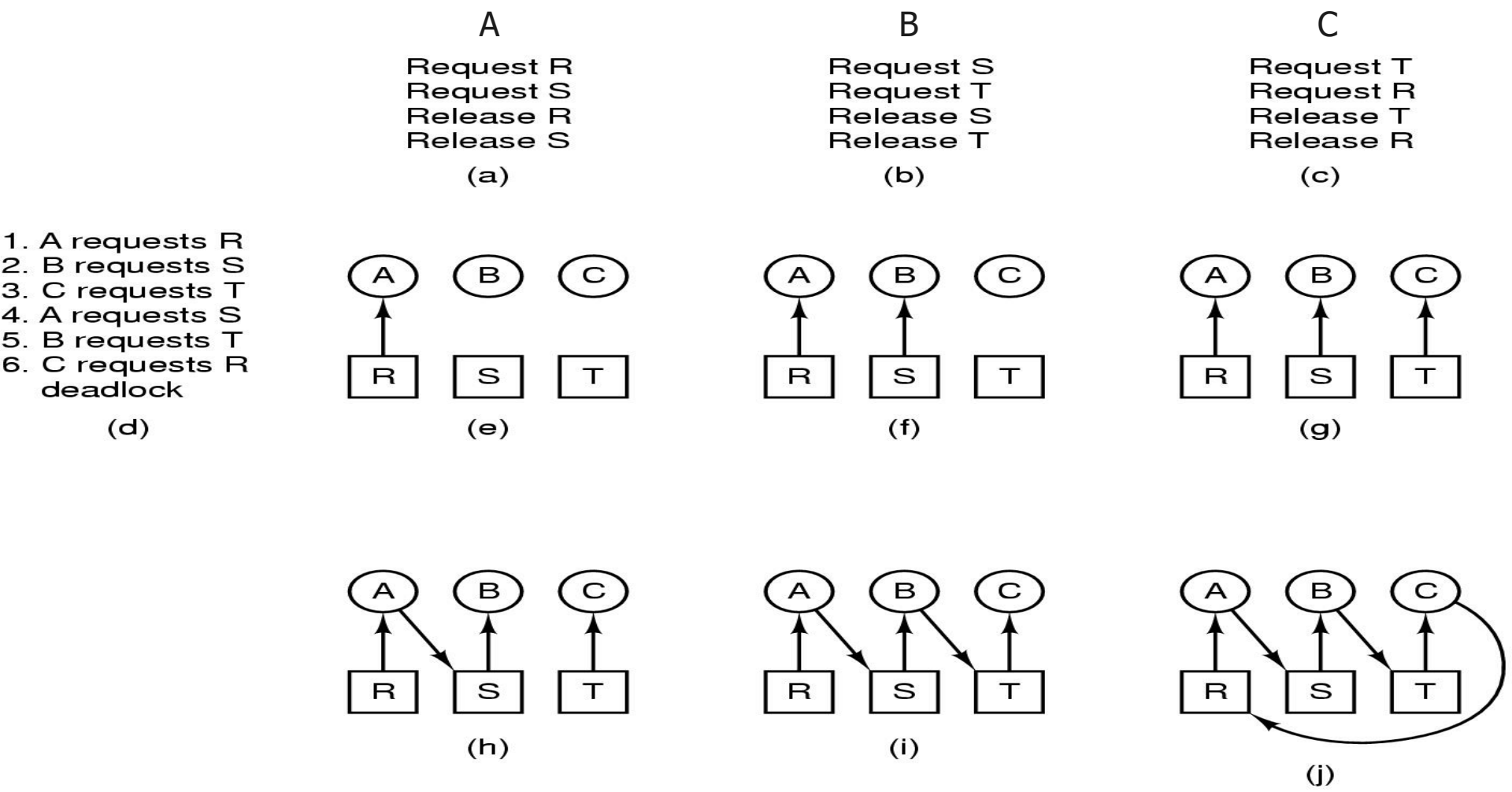
Resource Allocation Graph With a Cycle But No Deadlock



Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock.
- If graph contains a cycle $\Rightarrow$
  - if only one instance per resource type, then deadlock.
  - if several instances per resource type, possibility of deadlock.

Deadlock Modeling



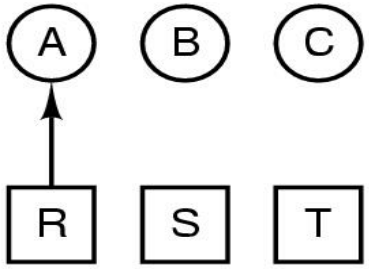
How deadlock occurs

Deadlock Modeling

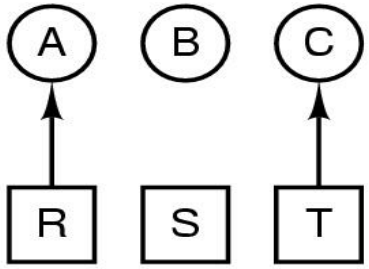
1. A requests R
2. C requests T
3. A requests S
4. C requests R
5. A releases R
6. A releases S

no deadlock

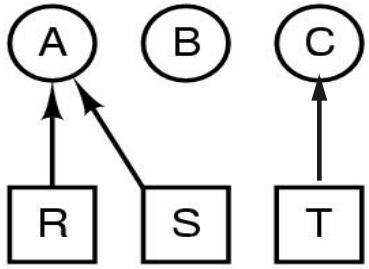
(k)



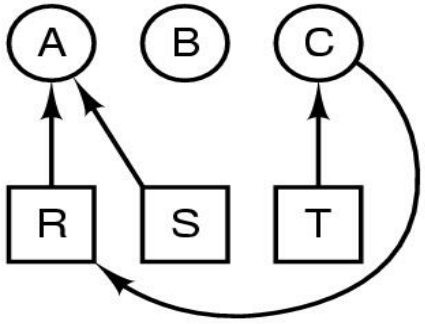
(l)



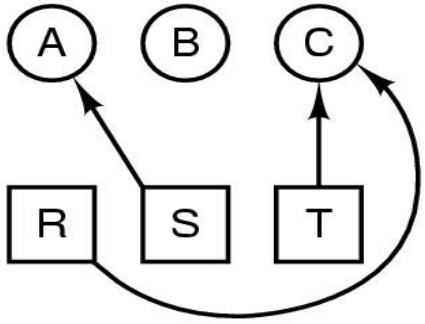
(m)



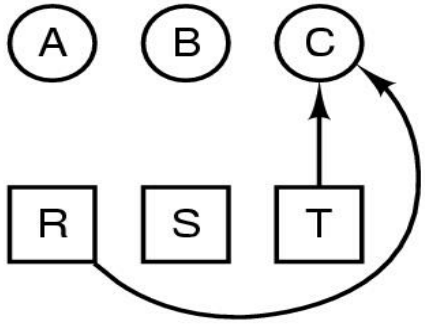
(n)



(o)



(p)



(q)

How deadlock can be avoided

Methods for Handling Deadlocks

1. Ignore the problem and pretend that deadlocks never occur in the system
2. Allow the system to enter a deadlock state and then recover.
   - detection and recovery
3. Ensure that the system will never enter a deadlock state
   - dynamic avoidance
     (careful resource allocation)
   - prevention
     (negating one of the four necessary conditions)

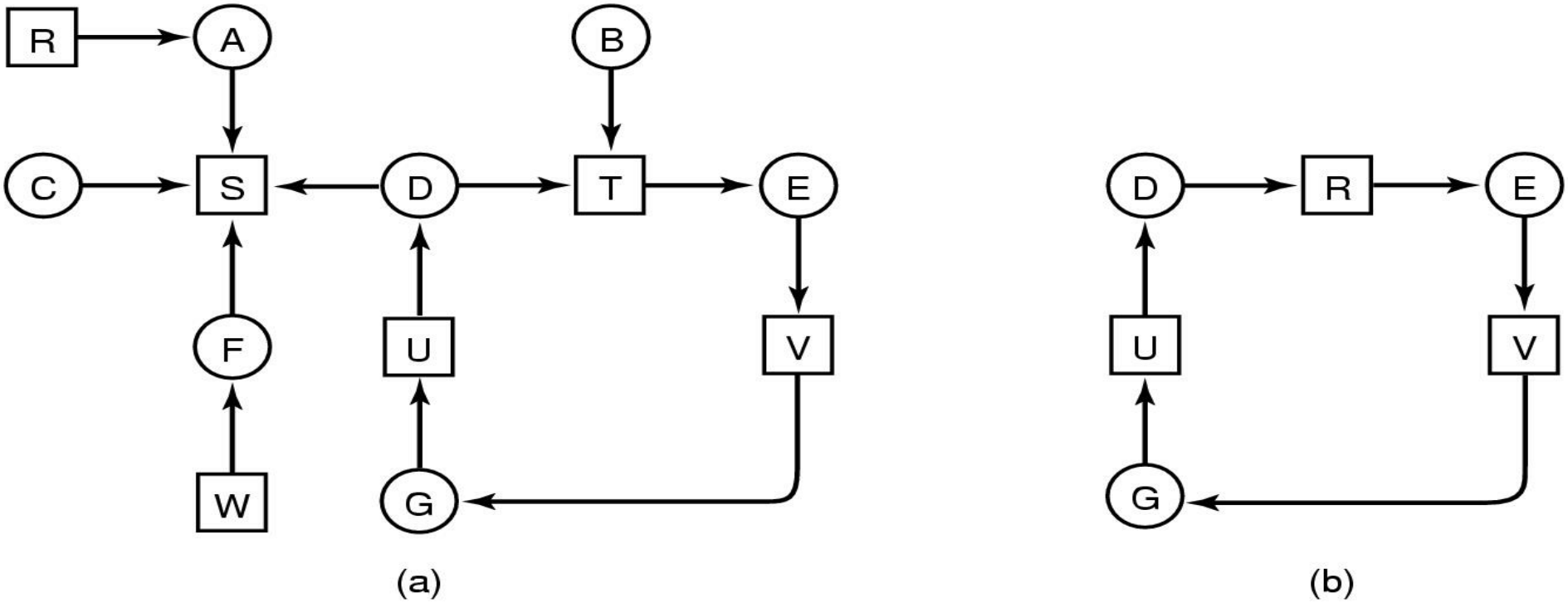
(1) The Ostrich Algorithm

- Pretend there is no problem
- Reasonable if
  - deadlocks occur very rarely
  - cost of prevention is high
- UNIX and Windows takes this approach
- It is a trade off between
  - convenience
  - correctness

(2) Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

Detection with One Resource of Each Type



- Note the resource ownership and requests
- A cycle can be found within the graph, denoting deadlock

An algorithm for detecting cycles in directed graph

Periodically invoke an algorithm that searches for a cycle in the graph.

- A data structure, L, a list of nodes.
- Arcs will be marked to indicate that they have already been inspected, to prevent repeated inspections.

  For each node, N, in the graph, performing the following 5 steps with N as the starting node.

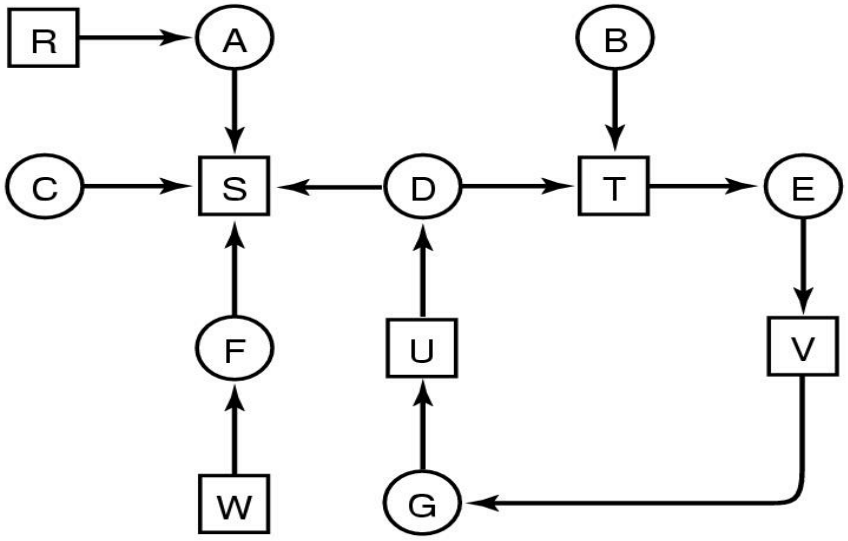
An algorithm for detecting cycles in directed graph

1. Initialize L to the empty list, and designate all the arcs as unmarked.
2. Add the current node to the end of L and check to see if the node now appears in L two times. If it does, the graph contains a cycle (listed in L) and the algorithm terminates.
3. From the given node, see if there are any unmarked outgoing arcs. If so, go to step 4; if not, go to step 5.
4. Pick an unmarked outgoing arc at random and mark it. Then follow it to the new current node and go to step 2.
5. We have now reached a dead end. Remove it and go back to the previous node, make it the current node, and go to step 3. If this node is the initial node, the graph does not contain any cycles and the algorithm terminates.

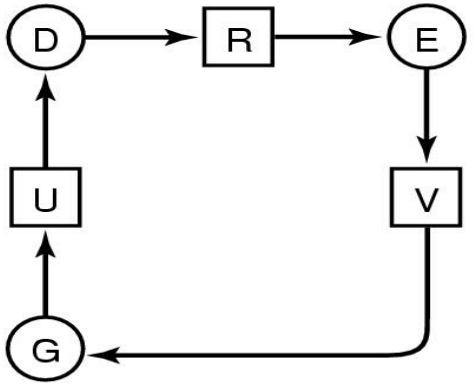
Example1

Start at R and initialize L to the empty list.

1. From R we go to S, giving L={R, A, S}. S has no outgoing arcs (a dead end).
2. Go backtrack to A.
3. A has no unmarked outgoing arcs, go backtrack to R.
4. R is the initial node, so no cycles.



(a)

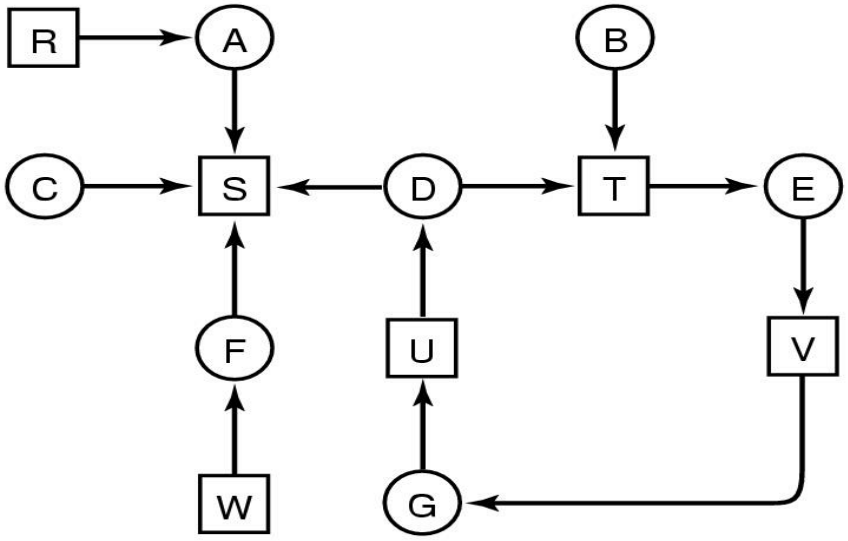


(b)

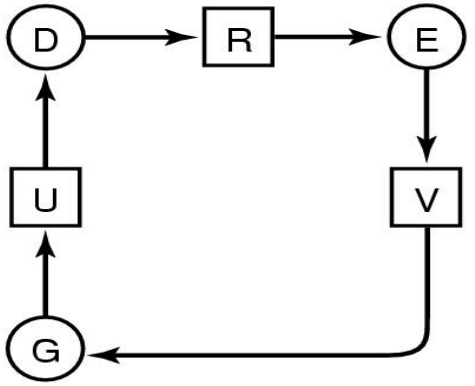
Example2

Start at B and initialize L to the empty list.

1. From B, we follow outgoing arcs until we get to D, giving L={B, T, E, V, G, U, D}.
2. Pick S randomly and we come to a dead end, then backtrack to D.
3. Now pick T and update L to be {B, T, E, V, G, U, D, T}. T appears two times in L, we discover cycle.



(a)



(b)

Detection with Multiple Resources of Each Type

Resources in existence
$(E_1, E_2, E_3, \ldots, E_m)$

Resources available
$(A_1, A_2, A_3, \ldots, A_m)$

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \cdots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \cdots & C_{nm} \end{bmatrix}$$

Row n is current allocation to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \cdots & R_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \cdots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2 needs

Data structures needed by deadlock detection algorithm

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

Deadlock Detection Algorithm

- Based on comparing vectors.

  (A<B hold if and only if Ai < Bi for 1<=i<=m)
- Each process is initially unmarked.

1. Look for an unmark process Pi, for which the i-th row of R is less than or equal to A. (Pi's request can be satisfied)
2. If such a process is found, add the i-th row of C to A, mark the process and go back to step 1. (When Pi completes, its resources will become available).
3. If no such process exist, the algorithm terminates. The unmarked process, if any, are deadlock.

Detection with Multiple Resources of Each Type (Example)

Tape drives Plotters Scanners CD Roms

$E = (4 \quad 2 \quad 3 \quad 1)$

Tape drives Plotters Scanners CD Roms

$A = (2 \quad 1 \quad 0 \quad 0)$

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Request matrix

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

An example for the deadlock detection algorithm

Recovery from Deadlock

- Recovery through preemption
  - take a resource from some other process
  - depends on nature of the resource
- Recovery through rollback
  - checkpoint a process periodically
  - use this saved state
  - restart the process if it is found deadlocked

Recovery from Deadlock

- Recovery through killing processes
  - crudest but simplest way to break a deadlock
  - kill one of the processes in the deadlock cycle, then the other processes get its resources
  - kill process that can be rerun from the beginning with no ill effects